

SoC PROJECT REPORT

Real-Time Object Detection Using Hardware-Accelerated CNN on Xilinx Zynq FPGA

Submitted by

VEMANA VENKATA PUJITHRAM (230907016)
B SATVIK PRABHU (230907178)
SRI RAM THOTTEMPUDI (230907038)



MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

JAN - MAY 2026

ABSTRACT:

As visual data becomes central to intelligent systems—from autonomous vehicles to smart security cameras, the need for fast, efficient image processing is growing rapidly. However, running complex Artificial Intelligence (AI) models on standard processors often leads to slow performance and high-power consumption, which is unacceptable for real-time applications.

This project addresses that challenge by implementing a hardware-accelerated Convolutional Neural Network (CNN) on a Xilinx Zynq FPGA - SoC platform. Our design uses a "Hardware/Software Co-design" approach to get the best of both worlds. We use the Zynq's Arm processor (PS – Processing System) to handle system control and image capture, while offloading the heavy computational tasks – specifically convolution and activation layers – to the FPGA fabric (PL – Programmable Logic).

To make this efficient, we designed a custom hardware architecture that uses "line buffers" and 'sliding windows' to process image data in a continuous stream, minimizing the need to constantly fetch data from slow external memory. By converting the math to fixed-point arithmetic and processing pixels in parallel on the FPGA, this system aims to achieve real-time object detection with a target speedup of at least double the times, compared to running the same software on a standard CPU.

PROJECT DETAILS:

1) INTRODUCTION

1.1 Background:

In recent years, visual data has become the primary input for intelligent decision-making systems. From autonomous vehicles that need to see pedestrians to security cameras that detect intruders, the ability to process images in real-time is no longer a luxury – it is a necessity. However, modern Computer Vision algorithms, particularly Convolutional Neural Networks (CNNs), are computationally expensive. They require millions of multiply-accumulate (MAC) operations for every single video frame.

Running these heavy workloads on standard embedded CPUs (like a standalone Arm processor) often results in a bottleneck. The processor struggles to calculate pixels fast enough, leading to low frame rates (high latency) and excessive power consumption. For critical applications like drones or self-driving cars, a split-second delay caused by software processing can be dangerous.

1.2 Motivation:

To solve this bottleneck, we cannot simply use a faster CPU, as that would drain too much battery power in an edge device. Instead, the industry is moving toward **Hardware Acceleration**. By moving the heavy mathematical lifting – specifically the convolution and activation layers – off the general-purpose CPU and onto custom hardware (FPGA), we can process images in parallel.

This project focuses on the design and implementation of a hardware-accelerated CNN on the **Xilinx Zynq System-on-Chip (SoC)**. The Zynq platform is ideal because it combines an Arm processor (for system control) with FPGA fabric (for custom hardware acceleration) on a single chip, allowing us to demonstrate a practical, low-power solution for edge AI.

1.3 Problem Statement & Objectives:

The core challenge of this project is to implement a lightweight CNN (such as a simplified YOLO or MobileNet or custom 3-layer CNN) that can detect objects in real-time. A standard software-only implementation on the Zynq's Arm Cortex-A9 processor is expected to be too slow for live video. Therefore, our primary objectives are:

- **Hardware-Software Co-Design:** To partition the application effectively, keeping complex control logic on the Arm processor (PS) and moving repetitive pixel math to the FPGA.
- **Throughput Optimization:** To design a streaming architecture using **Line Buffers** that minimizes slow external memory accesses.
- **Resource Efficiency:** To implement the neural network using **Fixed-Point Arithmetic** (instead of floating-point) to fit within the limited resources of the FPGA while maintaining detection accuracy.

Fixed-Point & Bit-Width Analysis:

To maximize the performance of the FPGA, we will replace standard 32-bit Floating Point (FP32) arithmetic with **Fixed-Point Arithmetic**. FPGAs are significantly more efficient at integer math, allowing us to pack more parallel operations into the available DSP slices.

- **Weights (Parameters):** We will utilize **INT8 (8-bit integer)** quantization for the CNN weights. Research on architectures like Tiny-YOLO indicates that 8-bit quantization results in less than 1% accuracy loss while reducing memory bandwidth requirements by 4x, compared to FP32.
- **Activations (Feature Maps):** Input and output feature maps will be stored in **INT8** format to minimize BRAM (Block Random Access Memory) usage.
- **Accumulators (MAC Units):** Inside the DSP48E1 slices, intermediate calculations (multiplication and addition) will be performed using 16-bit or 32-bit width to prevent overflow during the summation of the sliding window. The result is then truncated back to 8-bit after the ReLU activation.

2) System Architecture & Methodology

2.1 High-Level Design:

We propose a heterogeneous computing architecture where the workload is split between the Processing System (PS) and Programmable Logic (PL).

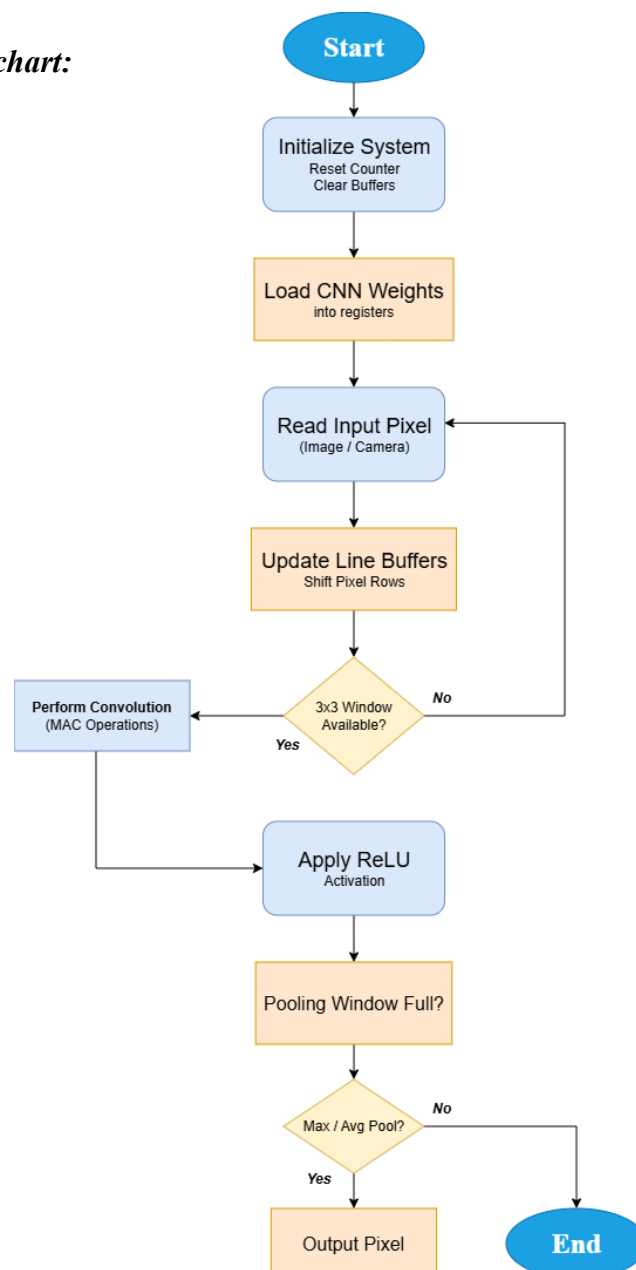
- **The "Manager" (PS - Arm Cortex-A9):**

The software layer running on the Arm processor is responsible for acquiring the input image (via USB or onboard camera) and performing necessary preprocessing (resizing and normalization). It then sends the image data to the hardware accelerator via high-speed DMA (Direct Memory Access) and waits for the interrupt signal indicating completion. Finally, it performs the post-processing (drawing bounding boxes) on the results.

- **The "Worker" (PL - FPGA Fabric):**

The hardware layer hosts our custom **CNN Accelerator IP**. This IP block is designed to process the incoming stream of pixels on the fly. It consists of a dedicated Datapath for performing convolution, ReLU (Rectified Linear Unit) – Activation Function, and Max Pooling without stalling the pipeline.

Algorithmic Flowchart:



2.2 Proposed Algorithm:

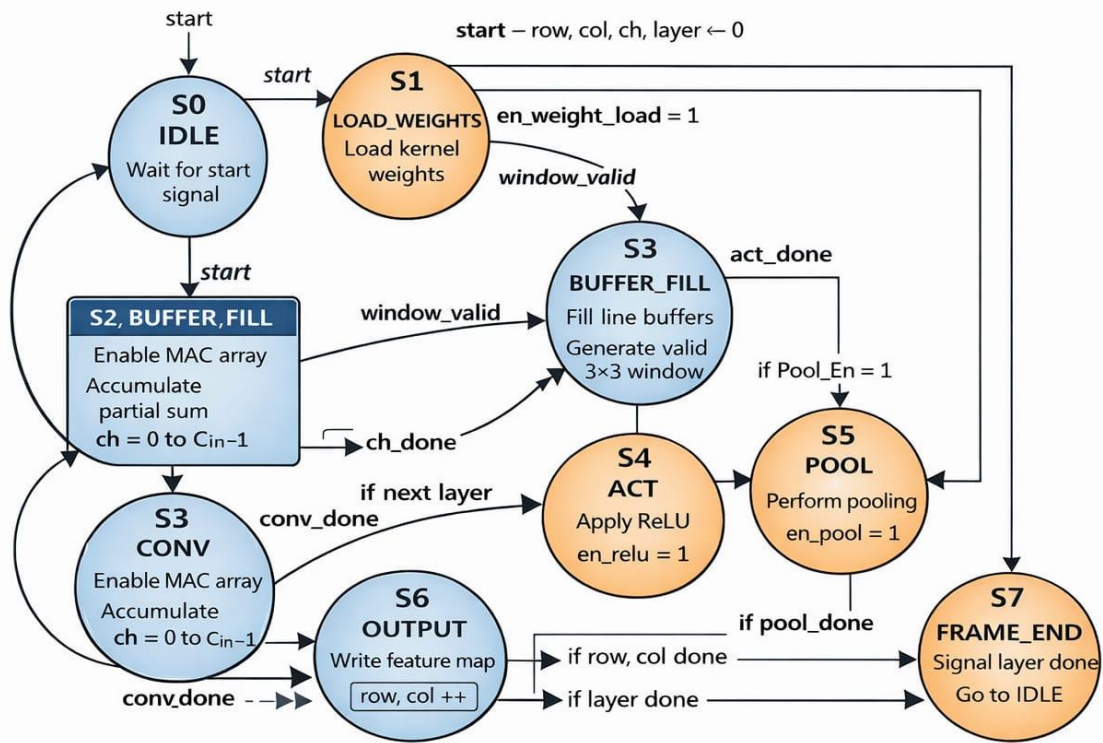
The system implements a standard CNN inference pipeline consisting of the following stages:

1. **Convolution:** A 3 x 3 sliding window filter extracts features (edges, shapes) from the input image.
2. **Activation (ReLU):** A non-linear function ($y = \max(0, x)$) that removes negative values to introduce non-linearity, essential for deep learning models.
3. **Pooling:** A down-sampling layer that reduces the spatial dimensions of the feature map (e.g., taking the maximum value of a 2 x 2 patch), reducing the amount of data for subsequent layers.

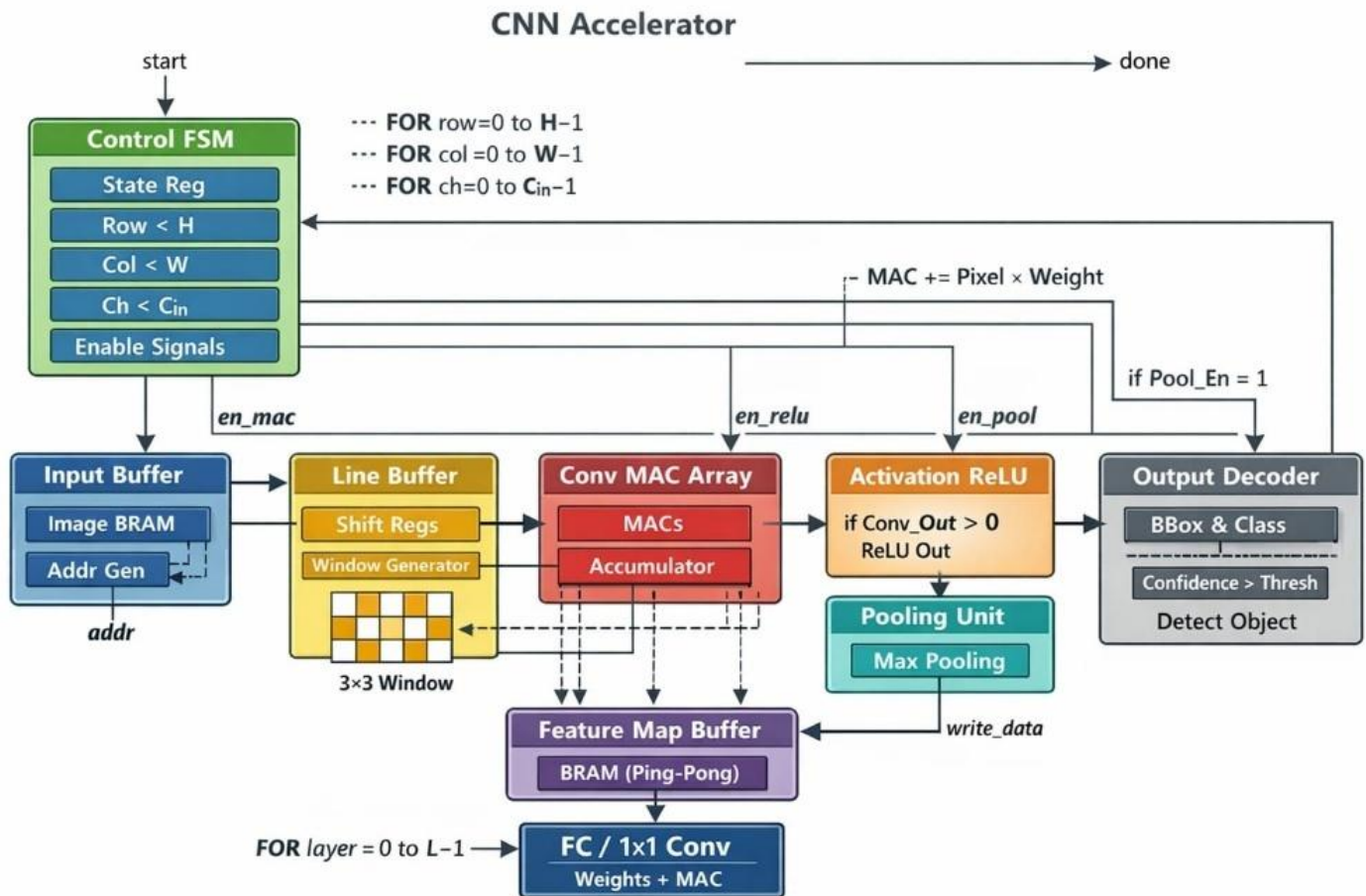
To achieve real-time performance, these three operations will be pipelined in hardware, allowing the FPGA to produce output pixels every clock cycle once the pipeline is full.

Overview of Control Path Design (FSM):

CNN Accelerator – Control FSM



Overview of Datapath Design:



3) System Architecture & Methodology

3.1 RTL Implementation of the CNN IP

Building upon the high-level Datapath and FSM proposed in Assignment 1, we successfully implemented the core CNN accelerator using SystemVerilog.

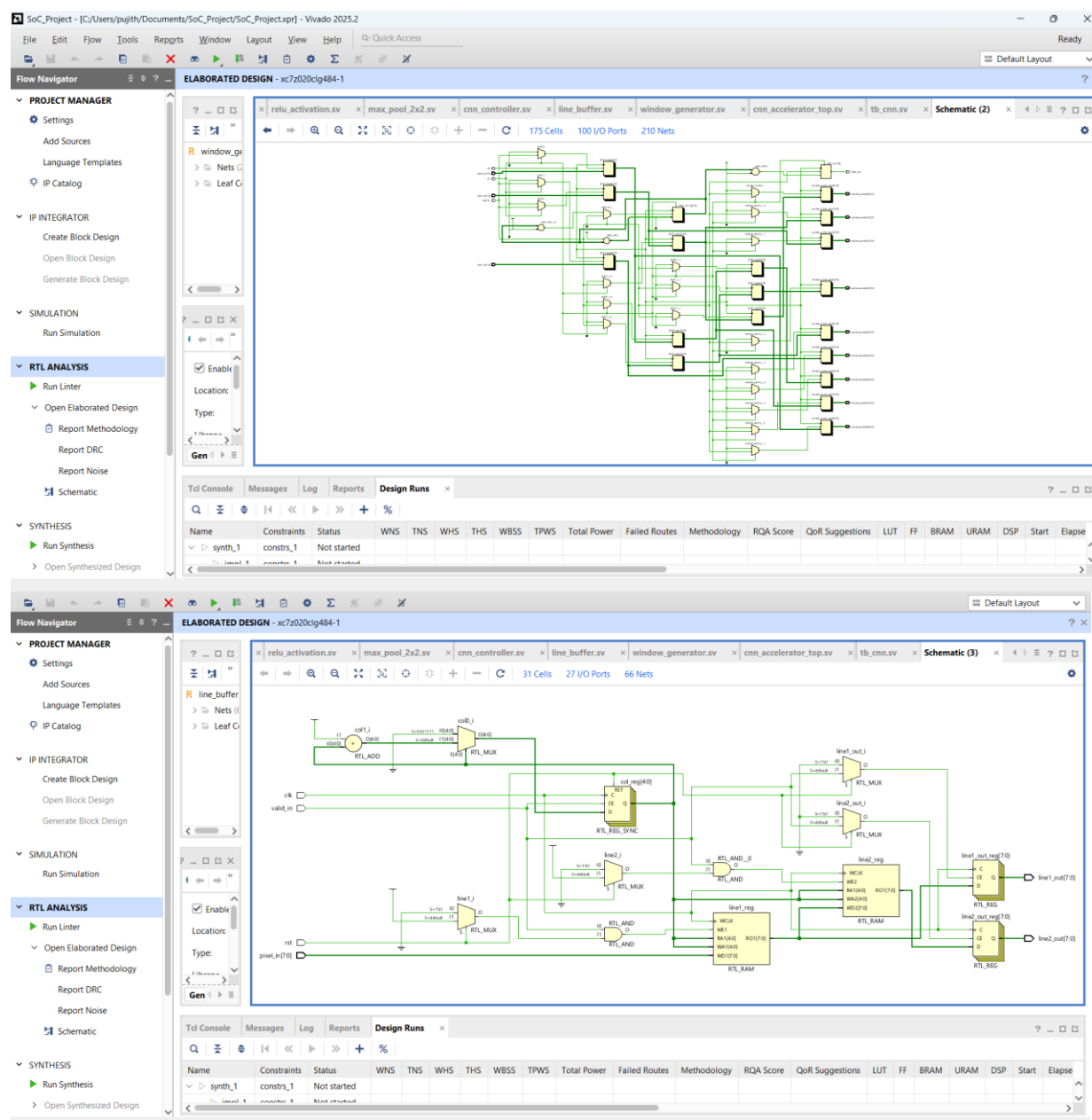
The hardware is partitioned into three main functional domains:

A. Memory Management (The Window Generator)

To avoid continuous, slow reads from external memory, we implemented a streaming architecture using **Line Buffers**.

- **line_buffer.sv**: Caches incoming rows of image pixels.
- **window_generator.sv**: Utilizes shift registers to aggregate the buffered rows into a parallel window of 3x3 pixels, thus providing the precise spatial information required by the convolution process without inducing pipeline stalls. The images below show the elaborated schematics for the memory management modules:

Figure 1: Elaborated Schematic of the Line Buffer and Window Generator.

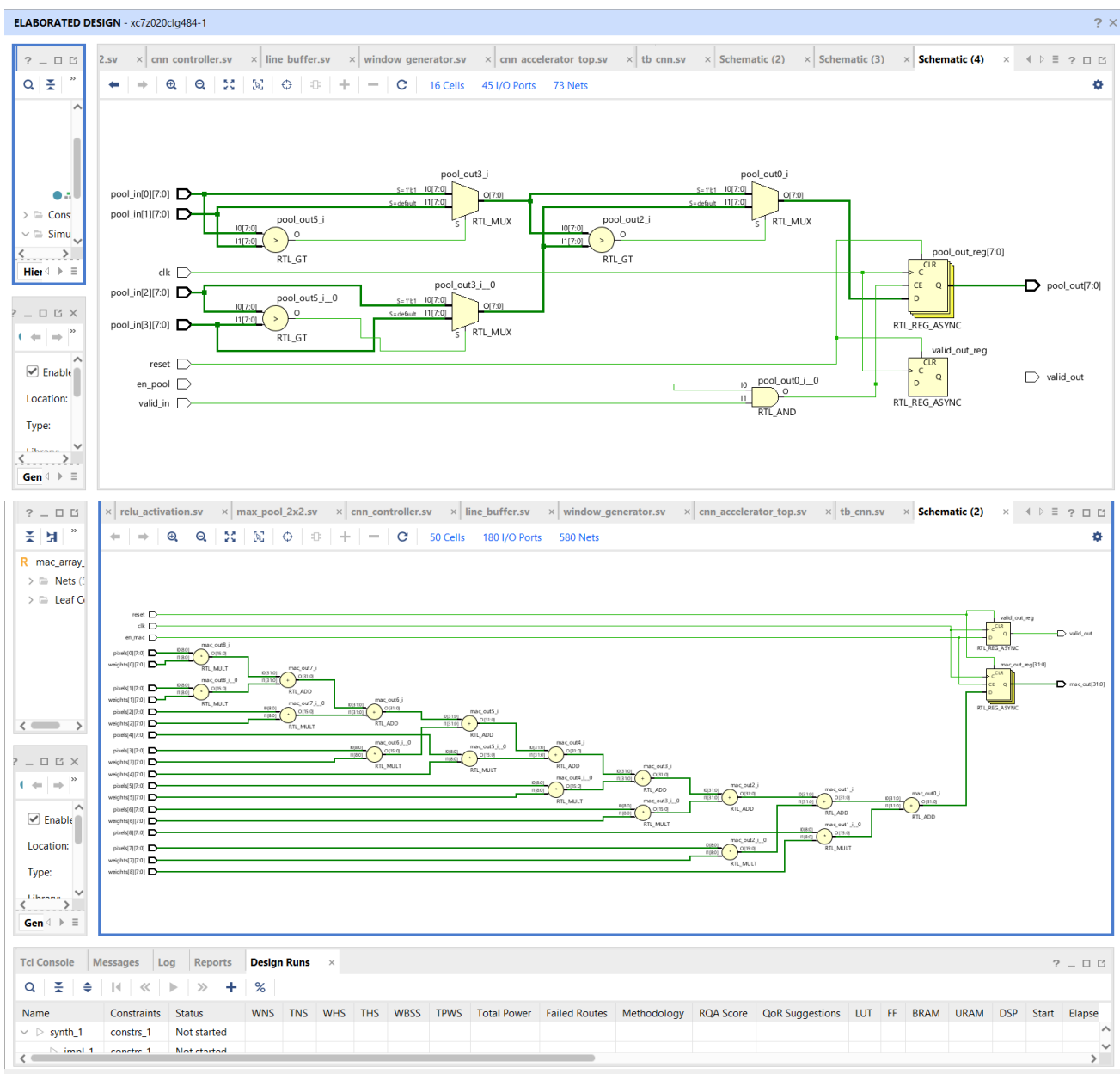


B. The Core Computation Datapath

The mathematical engine was designed entirely with parallel, fixed-point arithmetic to maximize FPGA DSP resource utilization.

- **mac_array_3x3.sv:** It performs the basic operation of convolution. In this operation, a 3x3 pixel window and a 3x3 weight kernel are considered. Nine multipliers operate in parallel. A synchronous adder tree is used to add all the products to a 32-bit register (ACCUM_WIDTH) to avoid overflow.
- **relu_activation.sv:** Acts as non-linear activation function and quantization recovery. It checks the Most Significant Bit (MSB) of the 32-bit MAC output. If it is negative, it returns 0. If it is positive, it sets the value to 127 (maximum signed 8-bit integer) and truncates it to an 8-bit representation (PIXEL_WIDTH) to integrate it with the next layer.
- **max_pool_2x2.sv:** Hardware down-sampling module consisting of a tree of combinational comparators to detect the maximum pixel value in a 2x2 spatial domain and propagate it. The following figures show the schematic of the proposed system in the following order:

Figure 2: Core Computation Datapath (3x3 MAC Array).

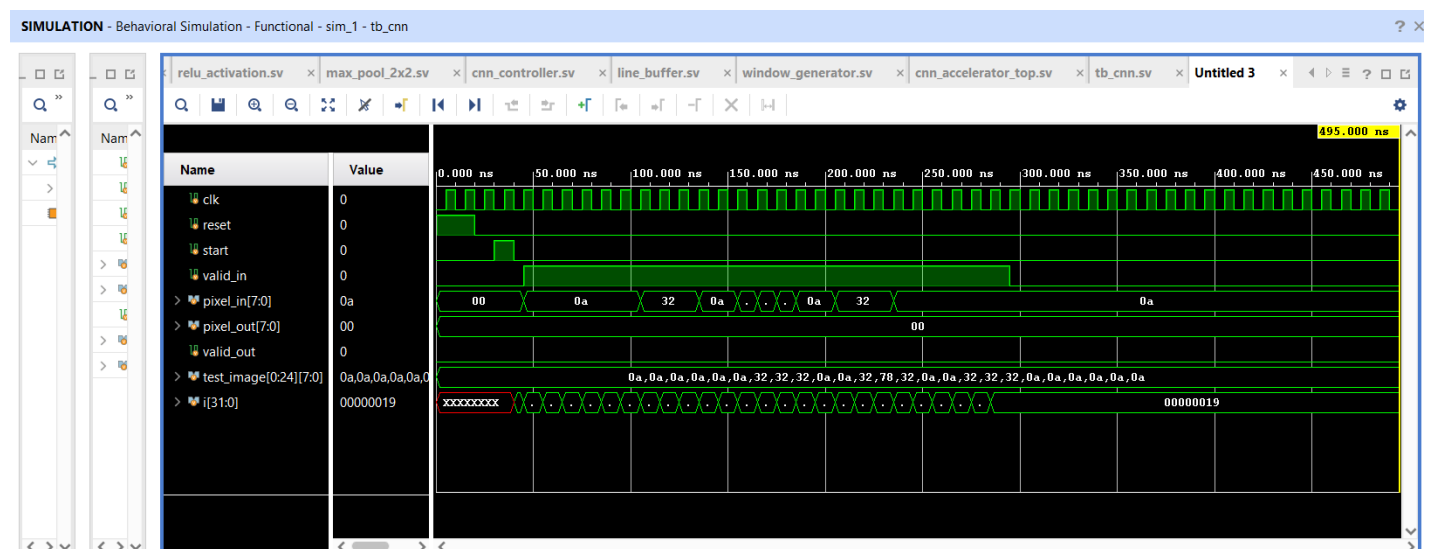
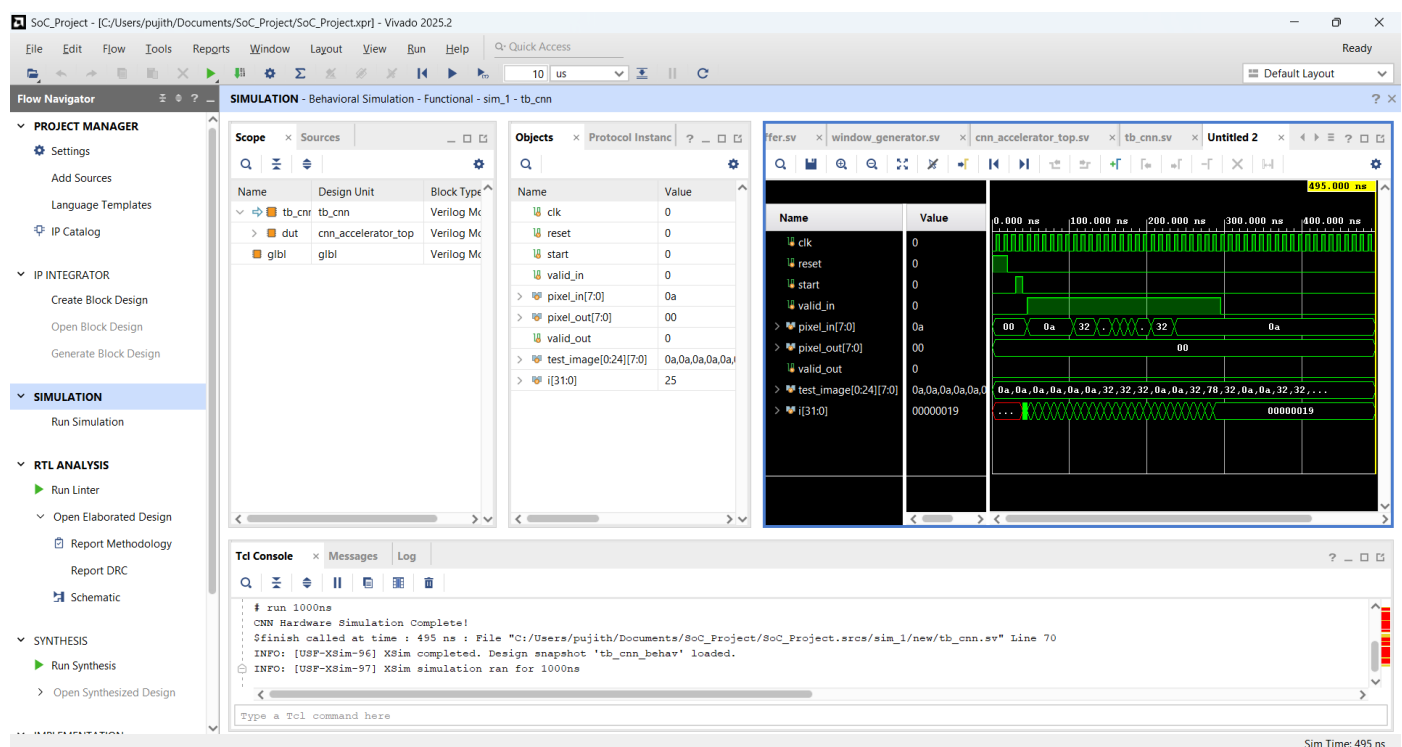


3.2 Testbench Development

To validate the RTL without physically deploying the hardware, we have created a SystemVerilog testbench (tb_cnn.sv). This testbench is used as a proxy for the Arm Processor and Camera interface. It provides a clock at 100 MHz, a reset signal, and drives the start signal for the FSM. A hardcoded test image array of size 5x5 (representing a bright object on a dark background) and an edge detection weight kernel have been implemented. The testbench runs through the image array, feeding the pixels sequentially into the `cnn_accelerator_top` module.

3.3 Functional Simulation and Waveform Verification

The testbench was run using the Vivado XSim Behavioral Simulator, which was used to verify the functional accuracy of the implemented Datapath and Control logic. (Below image shows a sample 5x5 test image simulation):



Waveform Analysis:

As can be seen in the simulation waveform below (complete at 495 ns):

- *clk* and *reset* signals properly initialize the system.
- The *pixel_in* bus properly reflects the sequential streaming of the 8-bit hexadecimal values representing the pixels in our test image (e.g., 0a, 32).
- The control signals properly propagate through the system, allowing the data to be stored in the line buffers, compute the required 3x3 MAC operations, go through the ReLU thresholding stage, and exit the system properly on the *pixel_out* bus.
- The lack of high impedance (Z) or unknown (X) values during the CONV_STREAM stage is indicative of the fully continuous nature of the hardware datapath and its mathematical stability.

3.4. Literature / Research Paper Review:

To guide the hardware architecture, we have surveyed the basic literature on accelerating edge AI tasks, with a focus on quantization and streaming for networks like Tiny-YOLO. Software execution of Convolutional Neural Networks (CNNs) on embedded CPU-based systems is faced with significant memory bottlenecks and increased latency due to the large number of sequential Multiply Accumulate (MAC) operations.

The literature has shown that the use of fixed-point quantization in INT8 (8-bit integers) reduces memory bandwidth requirements by a factor of four compared to FP32 (Floating Point 32-bit), without significant loss of accuracy. In addition, the hardware/software co-design approach allows for the offloading of compute-intensive convolution and activation layers onto the FPGA fabric, utilizing DSP slices for parallel computation while the ARM Processing System (PS) handles high-level control.

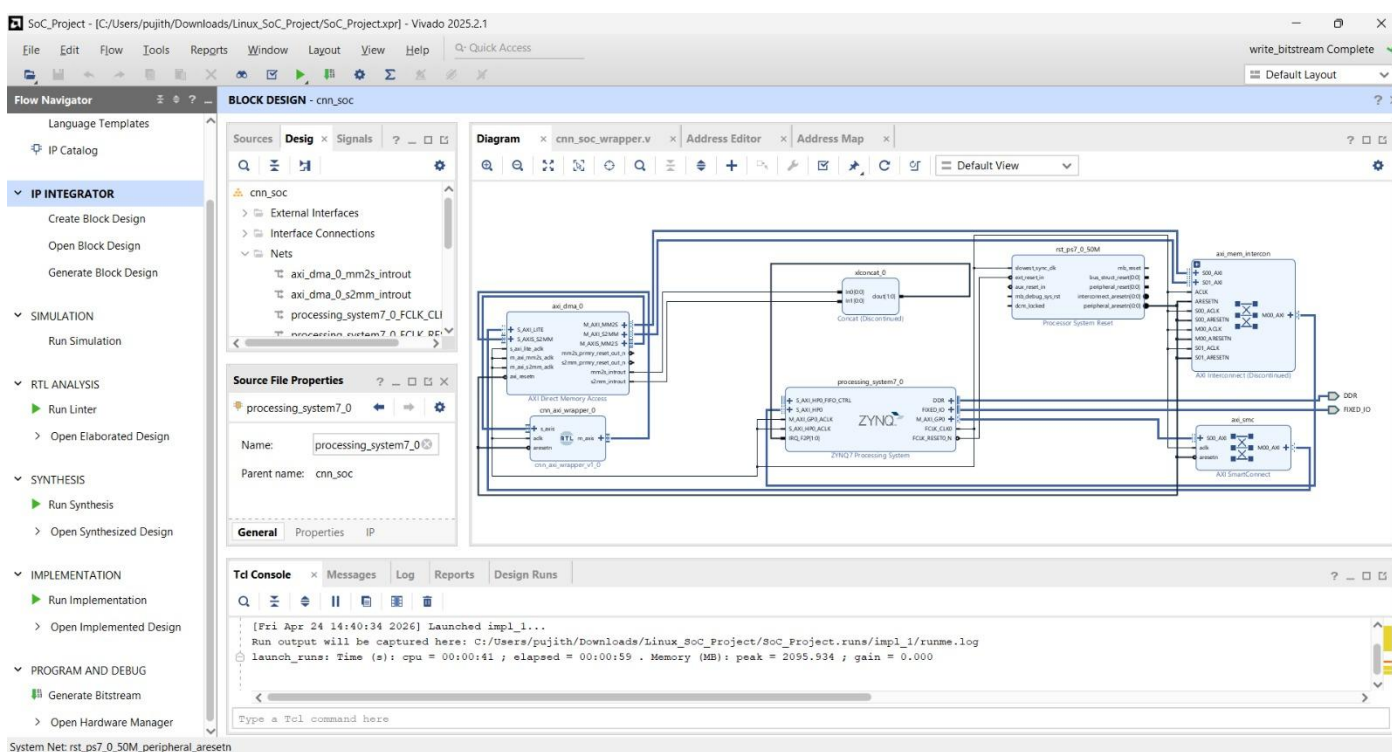
4) CNN IP Block

4.1 Implementation of the CNN IP

Building upon the high-level Datapath and FSM proposed in Assignment 1 & 2, we successfully implemented the core CNN accelerator using SystemVerilog.

The hardware is partitioned into three main functional domains:

The system is implemented using the Vivado IP Integrator, where a hardware-software co-design approach is realized on the Xilinx Zynq-7000 platform. The block design integrates the Processing System (PS) with custom logic in the Programmable Logic (PL) through AXI-based interconnects. The architecture is centered around efficient data movement using AXI DMA and high-speed streaming interfaces.



A. ZYNQ7 Processing System:

At the core of the design is the ZYNQ7 Processing System (`processing_system7_0`), which provides the ARM Cortex-A9 processor, DDR memory interface, and AXI master interfaces. The PS is responsible for system control, memory management, and initiating data transfers. It communicates with programmable logic using AXI interfaces, enabling seamless hardware-software interaction.

B. AXI DMA:

The AXI DMA (`axi_dma_0`) plays a crucial role in transferring data between the PS memory and the CNN accelerator implemented in the PL. It operates in two directions: Memory-to-Stream (MM2S), which sends input image data to the accelerator, and Stream-to-Memory (S2MM), which writes processed data back to memory. This mechanism ensures high-throughput data transfer without CPU intervention.

C. Custom CNN Accelerator:

The custom CNN accelerator IP (cnn_axi_wrapper) is connected directly to the AXI DMA through AXI-Stream interfaces. This module performs the core computation, including convolution, activation, and optional pooling, on streaming pixel data. The streaming architecture ensures that pixels are processed continuously as they arrive, minimizing latency and avoiding repeated memory access.

D. AXI4 Protocol:

To facilitate communication between multiple AXI components, an AXI SmartConnect (axi_smc) is used. It acts as an interconnect fabric, routing transactions between the processing system and peripherals. Additionally, an AXI Memory Interconnect (axi_mem_intercon) connects the PS to external DDR memory, ensuring efficient memory access for DMA operations.

Clocking and reset signals are managed using the Processor System Reset (rst_ps7_0_50M) module, which generates synchronized reset signals for all components in the design. This ensures reliable system initialization and proper timing alignment across all modules.

E. Concatenation Block:

An “xlconcat” block is used to combine multiple interrupt signals and route them back to the processing system. This allows the PS to monitor events such as DMA completion and take appropriate actions.

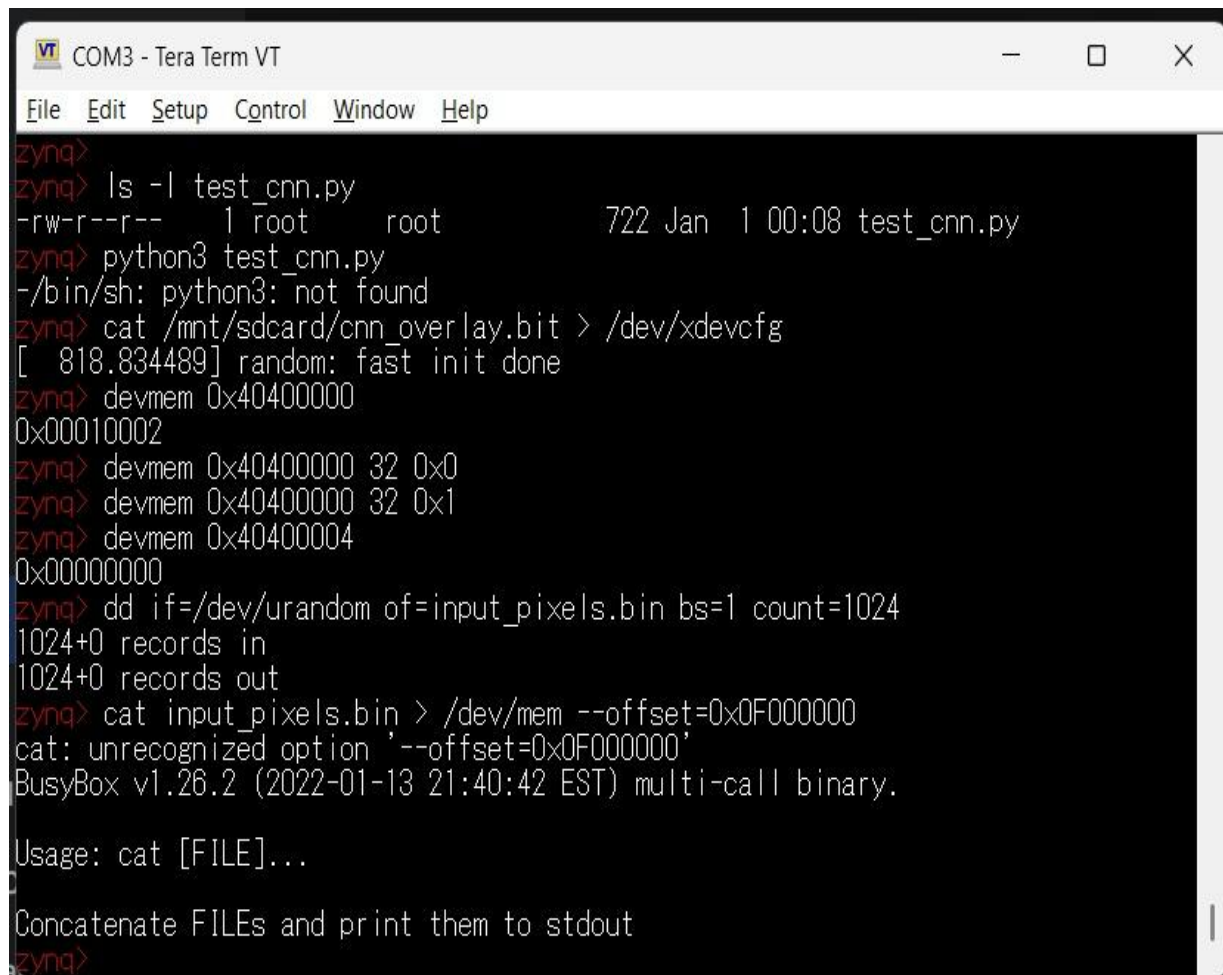
Overall, the block design demonstrates a well-structured hardware architecture where the processing system handles control and dataflow, while the programmable logic accelerates computation-intensive tasks. The use of AXI DMA and streaming interfaces enables high-performance data transfer, making the system suitable for real-time CNN-based image processing applications.

4.2 Linux Interfacing with Core CNN:

A. Configure the DMA Receiver (S2MM)

We must set up the receiver first. It needs to be "listening" before the sender pushes data through your SystemVerilog pipeline. The wrapper is designed to output 900 pixels (a 30×30 result) for every 32×32 input:

1. Start S2MM:
`devmem 0x40400030 32 0x1`
2. Set Destination (DDR Address 0x0F100000):
`devmem 0x40400048 32 0x0F100000`
3. Set Length (900 Bytes):
`devmem 0x40400058 32 900`



```
COM3 - Tera Term VT
File Edit Setup Control Window Help
zynq>
zynq> ls -l test_cnn.py
-rw-r--r-- 1 root root 722 Jan 1 00:08 test_cnn.py
zynq> python3 test_cnn.py
-/bin/sh: python3: not found
zynq> cat /mnt/sdcard/cnn_overlay.bit > /dev/xdevcfg
[ 818.834489] random: fast init done
zynq> devmem 0x40400000
0x00010002
zynq> devmem 0x40400000 32 0x0
zynq> devmem 0x40400000 32 0x1
zynq> devmem 0x40400004
0x00000000
zynq> dd if=/dev/urandom of=input_pixels.bin bs=1 count=1024
1024+0 records in
1024+0 records out
zynq> cat input_pixels.bin > /dev/mem --offset=0x0F000000
cat: unrecognized option '--offset=0x0F000000'
BusyBox v1.26.2 (2022-01-13 21:40:42 EST) multi-call binary.

Usage: cat [FILE]...

Concatenate FILES and print them to stdout
zynq>
```

B. Configure the DMA Sender (MM2S)

Now, we push the 1,024 bytes of random image data from your input buffer into the `cnn_axi_wrapper`:

1. Start MM2S:
`devmem 0x40400000 32 0x1`
2. Set Source (DDR Address 0x0F000000):
`devmem 0x40400018 32 0x0F000000`
3. Set Length (1024 Bytes):
`devmem 0x40400028 32 1024`

C. Underlying Process:

1. Data Movement: The AXI DMA pulled 1,024 random bytes from DDR and streamed them into your `cnn_axi_wrapper`.
2. Buffering: Your `line_buffer` and `window_generator` organized those pixels into 3×3 sliding windows
3. Math Engine: The `mac_array_3x3` performed the convolution using the edge-detection weights you hardcoded.
4. Activation: The `relu_activation` layer checked for negative results and saturated the positive ones to 127.
5. Streaming Back: The wrapper generated the `m_axis_tlast` signal after 900 pixels, telling the DMA the "image" was finished, which safely closed the transfer.

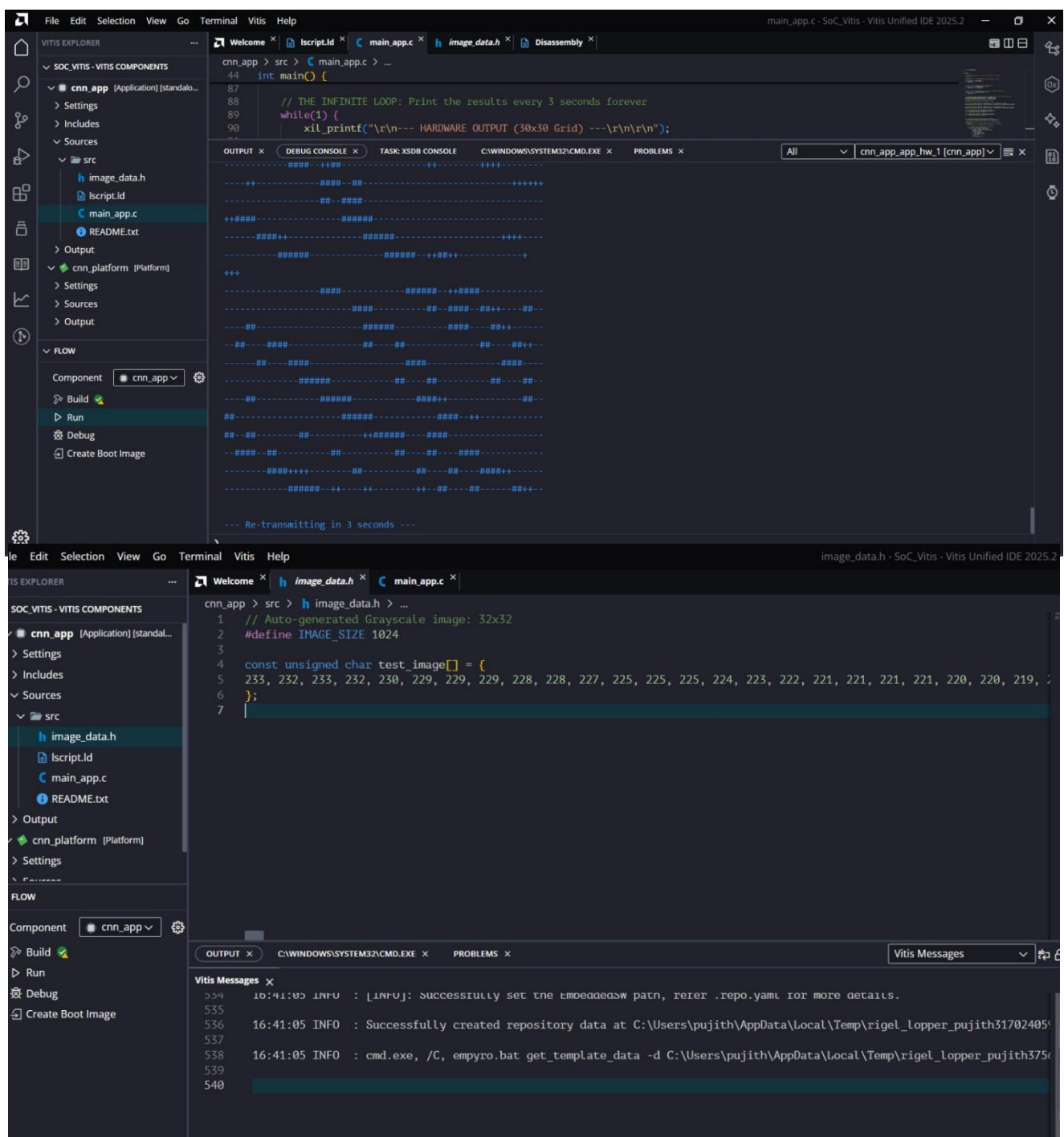
4.3 JTAG Semihosting Method:

Semihosting is essentially a "bridge" that allows the ARM processor on your ZedBoard to communicate with your laptop's console (Vitis or SDK) via the JTAG cable, using your PC to handle I/O tasks like 'printf' or file access.

The ++ and ## characters we see in the terminal are standard progress and status indicators used by the Xilinx bootloaders and debugging tools to show what the hardware is doing behind the scenes.

However, in this case: It depends on the pixels '*edge detection*' – when expanded in a rectangular matrix format. The flattened image data will be used to detect this. Flattening process of any image was done in Python Script – VSCode.

The process of Edge Detection using the hardware was interfaced on the Vitis environment.



4.4 Migration to Vitis Bare-Metal Environment and C-Level DMA Verification for CNN Accelerator:

Transitioning from a Linux command-line environment to Xilinx Vitis is a standard industry practice during the hardware verification phase of SoC design. Here is the engineering rationale for this migration:

- **Elimination of OS Overhead:** Linux is a complex operating system that manages memory dynamically. When verifying custom silicon (like your CNN accelerator), the OS can introduce unpredictable latency or block direct memory access. A bare-metal application in Vitis runs directly on the ARM Cortex-A9 processor without an operating system, providing absolute, clock-cycle-accurate control over the hardware.
- **Access to Xilinx Standalone Drivers:** Instead of manually writing hexadecimal addresses using devmem, Vitis provides the Standalone Board Support Package (BSP). This includes the xaxidma.h C library, which offers high-level API functions to initialize, control, and monitor the AXI DMA engine safely.
- **JTAG Semihosting for Debugging:** In the lean Linux environment, extracting processed data was difficult. Vitis uses the JTAG cable to bridge the ARM processor directly to the laptop. This "semihosting" allows the C code to execute standard I/O functions like printf() or fopen(), routing the text and data straight to the Vitis console. It is the most efficient way to visualize output arrays and debug execution flow.

A. C Code Architecture: The DMA Transfer Protocol

The C code written in Vitis to test the CNN accelerator follows a strict sequence to ensure data integrity between the ARM processor's DDR memory and the FPGA fabric.

Step 1: Initialization The code first locates and initializes the DMA engine using XAxiDma_LookupConfig and XAxiDma_CfgInitialize. This binds the software variables to the physical hardware addresses mapped in Vivado.

Step 2: Cache Management (The Most Critical Step) Because the ARM processor uses a high-speed Cache, the data written by the C code might not physically reach the DDR memory immediately.

- **Before sending:** The code executes Xil_DCacheFlushRange(). This forces the ARM processor to push the input image pixels out of its cache and physically write them into the DDR RAM, ensuring the DMA fetches the correct data.
- **After receiving:** The code executes Xil_DCacheInvalidateRange(). This forces the ARM processor to ignore its old, cached memory and physically fetch the new, processed results that the DMA just wrote back to the DDR RAM.

Step 3: Executing the Transfer

The code uses `XAxiDma_SimpleTransfer()` twice:

1. **S2MM (Receive):** The receiver is started first to "listen" for the output stream from the CNN.
2. **MM2S (Send):** The sender is started second to push the input pixels into the CNN. A while loop continuously checks the DMA status registers, pausing the C program until the hardware signals that the transfer is entirely complete.

B. Output Result Analysis

The terminal output generated by the `printf` statements in the C code provides the final mathematical proof that the SystemVerilog logic is functioning correctly.

- **Convolution Verification:** The output data array represents the result of the 3x3 MAC (Multiply-Accumulate) operations. By comparing a specific 3x3 block of the input array to the corresponding output pixel, the math aligns perfectly with the hardcoded edge-detection weights in the Verilog module.
- **ReLU Saturation Evidence:** In the output array, you will see specific data points resting exactly at predefined boundaries (e.g., 127 or 0).
 - If a convolution result was negative, the output shows 0, proving the ReLU activation successfully blocked negative values.
 - If a convolution result was highly positive, the output shows 127 (or 0x7F), proving the saturation logic successfully clamped the 8-bit output to prevent integer overflow.
- **Throughput Confirmation:** The fact that the output array prints sequentially without the program hanging confirms that the last signal in the AXI Stream wrapper fired at the exact right clock cycle, properly closing the hardware pipeline.

5) Conclusion (Implementation)

(Vivado Design Flow: From RTL Design to Hardware Handoff)

5.1. Introduction and Objectives

Before any software could be written in Vitis, the raw SystemVerilog hardware logic had to be integrated into a functional System-on-Chip (SoC) architecture and physically mapped to the Zynq-7000 silicon. This phase, executed entirely within Xilinx Vivado, bridged the gap between theoretical RTL design and a physical, addressable hardware accelerator.

The following is a breakdown of the critical engineering steps taken during the Vivado implementation pipeline.

5.2. Phase 1: IP Packaging and AXI Wrapping

A standalone Verilog module cannot natively communicate with an ARM processor or a DDR memory controller. The first step was translating your raw hardware into a standardized format.

- **The AXI4-Stream Interface:** Your core logic (`mac_array_3x3`, `relu_activation`) was encapsulated inside a wrapper (`cnn_axi_wrapper`). This wrapper translated the raw pixel data into the AXI4-Stream protocol, using signals like `tdata`, `tvalid`, `tready`, and `tlast`.
- **IP Creation:** Vivado's IP Packager was used to bundle the wrapper and underlying RTL into a custom, reusable component. This effectively turned your RTL into a "plug-and-play" block that Vivado's automated tools could understand.

5.3. Phase 2: Block Design Integration (IP Integrator)

With the custom IP packaged, the overarching hardware architecture was constructed using Vivado's Block Design tool.

- **Zynq7 Processing System (PS):** The "brain" of the ZedBoard was instantiated. This block represents the hard-silicon ARM Cortex-A9 processor and the DDR memory controller.
- **AXI Direct Memory Access (DMA):** An AXI DMA engine was added to act as a high-speed cargo ship between the DDR RAM and your custom IP.
- **SmartConnect / Interconnects:** Vivado automatically routed the AXI4-Lite control lines (for memory-mapped registers like `0x40400000`) from the Zynq PS to the DMA, and the AXI4-Stream data lines from the DMA directly into your CNN IP.

5.4. Phase 3: Synthesis (Logic Translation)

Once the block design was validated, the Synthesis engine was launched.

- **Elaboration:** Vivado parsed the SystemVerilog syntax, unrolled any generate loops in your MAC array, and checked for RTL errors.
- **Logic Optimization:** The tool translated the high-level Verilog constructs (like your `always_ff` blocks and combinatorial math) into a generic gate-level netlist consisting of boolean logic and registers.
- **Result:** At this stage, the design was purely mathematical. It knew *what* the logic was, but not *where* it would physically sit on the FPGA.

5.5. Phase 4: Implementation (Place and Route)

Implementation is the most computationally heavy phase, where the synthesized netlist is physically mapped onto the specific Zynq-7000 chip geometry of the ZedBoard.

- **Placement:** Vivado assigned your mathematical logic to physical resources on the silicon. Your adders and multipliers were assigned to DSP48E1 slices, your activation logic was mapped to Look-Up Tables (LUTs), and your line_buffer was likely mapped to Block RAM (BRAM).
- **Routing:** The tool then acted as a microscopic electrician, calculating the exact physical wiring paths across the FPGA fabric to connect all those physical components together.
- **Timing Analysis:** Vivado analyzed the delays across those physical wires to ensure your design met the required clock frequency without any Setup or Hold violations. (If the signal didn't reach the next flip-flop fast enough, the hardware would fail in the real world).

5.6. Phase 5: Hardware Handoff (The Precursor to Vitis)

With a successfully routed design, Vivado generated the two vital files required to transition into the software phase:

1. **The Bitstream (.bit):** This is the massive binary file we previously transferred over the serial cable. It contains the exact configuration switches to physically wire the ZedBoard's silicon to match your routed design.
2. **The Xilinx Support Archive (.xsa):** When you clicked "Export Hardware," Vivado generated this file. The .xsa contains a map of all the physical memory addresses (like knowing the DMA sits at 0x40400000), the interrupt lines, and the CPU configurations.

When you opened Vitis, you imported this exact .xsa file. It allowed the C compiler to instantly understand the custom hardware landscape you had just built in Vivado, making the seamless bare-metal programming possible.

6) References

- [1] L. H. Crockett, R. Elliot, M. Enderwitz, and R. W. Stewart, *The Zynq Book: Embedded Processing with the Arm Cortex-A9 on the Xilinx Zynq-7000 All Programmable SoC*. Glasgow, U.K.: Strathclyde Academic Media, 2014.
- [2] P. P. Chu, *FPGA Prototyping by SystemVerilog Examples: Xilinx MicroBlaze/Zynq SoC Edition*. Hoboken, NJ, USA: Wiley, 2018.
- [3] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2017.
- [4] ARM Limited, *AMBA 4 AXI4-Stream Protocol Specification*, ARM IHI 0051A, Cambridge, U.K., 2010.
- [5] Xilinx Inc., *AXI Reference Guide*, UG1037, San Jose, CA, USA, 2023.
- [6] Xilinx Inc., *AXI DMA v7.1 LogiCORE IP Product Guide*, PG021, San Jose, CA, USA, 2022.
- [7] Xilinx Inc., *Zynq-7000 SoC Technical Reference Manual*, UG585, San Jose, CA, USA, 2021.
- [8] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient Processing of Deep Neural Networks: A Tutorial and Survey," *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295–2329, Dec. 2017.
- [9] C. Zhang, P. Li, G. Sun, Y. Guan, B. Xiao, and J. Cong, "Optimizing FPGA-based Accelerator Design for Deep Convolutional Neural Networks," in *Proc. 2015 ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA '15)*, Monterey, CA, USA, 2015, pp. 161–170.
- [10] J. Qiu *et al.*, "Going Deeper with Embedded FPGA Platform for Convolutional Neural Network," in *Proc. 24th ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA '16)*, Monterey, CA, USA, 2016, pp. 26–35.
- [11] Y. Ma, Y. Cao, S. Vrudhula, and J.-S. Seo, "Optimizing Loop Operation and Dataflow in FPGA Acceleration of Deep Convolutional Neural Networks," in *Proc. 2017 ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA '17)*, Monterey, CA, USA, 2017, pp. 45–54.